

openclaw的指令

目前有三块系统：

- 1 OpenClaw Agent (AI核心)
- 2 VPN代理 (Gemini API网络出口)
- 3 Star Office UI (像素可视化)

很好，这一步其实很关键。你现在已经有 三块系统：

- 1 OpenClaw Agent (AI核心)
- 2 VPN代理 (Gemini API网络出口)
- 3 Star Office UI (像素可视化)

以后每次启动其实就是 按固定顺序启动三个组件。

我给你整理成一个**标准启动流程**（以后照着做就行）。

一、每次启动 OpenClaw 的完整流程

Step 1: 先打开 VPN

必须先开代理，不然 Gemini API 访问不到。

代理端口是：

代码块

```
1 7897
```

确认 VPN 已连接。

Step 2: 打开一个终端 (Terminal 1)

设置代理 + API key：

代码块

```
1 export https_proxy=http://127.0.0.1:7897
```

```
2 export http_proxy=http://127.0.0.1:7897
3 export all_proxy=http://127.0.0.1:7897
4 export NO_PROXY=127.0.0.1,localhost,::1
5
6 export GEMINI_API_KEY="xxxxxx"
```

然后启动 **OpenClaw Agent**

代码块

```
1 openclaw tui --local
```

这样你就会进入：

代码块

```
1 Claw 🦁 terminal
```

这里就是你的 **AI终端助手**。

二、启动像素可视化界面

再开一个终端（Terminal 2）。

进入项目目录：

代码块

```
1 cd ~/Star-Office-UI
```

启动 Python 虚拟环境：

代码块

```
1 source .venv/bin/activate
```

运行后端：

代码块

```
1 cd backend
```

```
2 python app.py
```

如果成功，会看到：

代码块

```
1 Listening on: http://0.0.0.0:19000
2 Running on http://127.0.0.1:19000
```

三、打开像素 UI

浏览器打开：

代码块

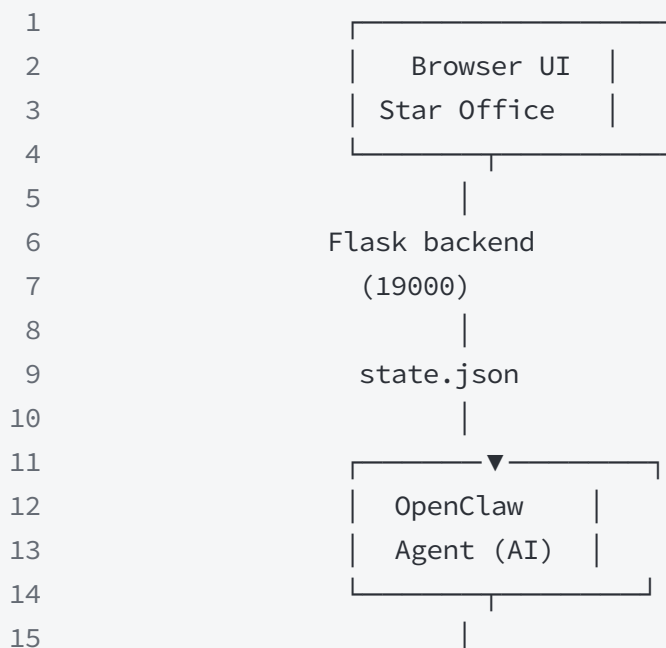
```
1 http://127.0.0.1:19000
```

你就会看到：

像素办公室界面

四、完整结构图（以后脑子里这样理解）

代码块



```
16 Gemini API
17 |
18 VPN
```

五、以后启动只需要记住这 6 条命令

终端1 (AI)

代码块

```
1 export https_proxy=http://127.0.0.1:7897
2 export http_proxy=http://127.0.0.1:7897
3 export all_proxy=http://127.0.0.1:7897
4 export GEMINI_API_KEY="你的key"
5
6 openclaw tui --local
```

终端2 (像素UI)

代码块

```
1 cd ~/Star-Office-UI
2 source .venv/bin/activate
3 cd backend
4 python app.py
```

浏览器

代码块

```
1 http://127.0.0.1:19000
```

六、下次我们可以做的升级（很有意思）

现在 UI 只是 读取 state.json。

下一步我们可以做三件事：

1 让 Star Office UI 自动读取 OpenClaw 状态

比如：

代码块

1

Claw 状态

2

当前任务

3

token 使用

2 做真正的像素 AI 办公室

让 Claw：

- 在桌子前工作
- 写代码
- 读论文
- 跑 IMU 数据

3 把你的研究 workflow 接进去

例如：

代码块

1

 论文阅读

2

 gait 数据分析

3

 实验记录

4

 GitHub 项目

最后一句结论

今天其实完成了一件很大的事情：

你已经搭好了一个：

本地 AI Agent + 像素可视化控制台

这在 AI 工具链里其实是 很高级的一种形态。

下次如果你愿意，我可以带你做一个非常酷的东西：

让 Claw 在像素办公室里 “真的动起来”。

比如：

- 思考时走来走去
- 运行代码时敲键盘
- 空闲时喝咖啡 ☕
- Debug 时抓狂 🦀

OpenClaw + Gemini + Star Office UI 本地 AI 工作台搭建与运行指南

作者：Asta

1 系统架构概览

本系统由三部分组成：

1. OpenClaw Agent（AI执行核心）
2. OpenClaw Gateway（后台服务通信层）
3. Star Office UI（像素风可视化界面）

整体结构：

Gemini API

|
| (VPN代理)

OpenClaw Agent

|
Gateway (后台服务 18789)

|
Star Office UI Backend (Flask)

2 环境准备

2.1 操作系统

推荐环境：

- macOS
- zsh shell

2.2 必要软件

安装以下工具：

- Node.js
- Python $\geq$ 3.11
- Git
- VPN / Clash (用于访问 Gemini API)

检查 Python 版本：

```
python3 --version
```

若低于 3.10，安装：

```
brew install python@3.11
```

3 安装 OpenClaw

安装：

```
npm install -g openclaw
```

检查版本：

```
openclaw --version
```

4 配置 Gemini API

在 Google AI Studio 创建 API Key。

建议注意：

⚠ 不要公开 API Key

⚠ 如果 Key 曾暴露，立即重新生成

5 配置系统环境变量

编辑 shell 配置：

```
nano ~/.zshrc
```

加入：

===== OpenClaw / Gemini =====

```
export https_proxy=http://127.0.0.1:7897
```

```
export http_proxy=http://127.0.0.1:7897
```

```
export all_proxy=http://127.0.0.1:7897
```

```
export NO_PROXY=127.0.0.1,localhost::1
```

```
export GEMINI_API_KEY="YOUR_KEY"
```

保存并执行：

```
source ~/.zshrc
```

验证：

```
echo $GEMINI_API_KEY
```

6 配置 OpenClaw 后台 Gateway

6.1 安装 LaunchAgent

```
openclaw gateway install
```

系统会创建：

```
~/Library/LaunchAgents/ai.openclaw.gateway.plist
```

6.2 启动后台服务

```
openclaw gateway start
```

验证端口：

```
lsof -i :18789
```


若看到 node LISTEN，说明 Gateway 已运行。

7 解决后台环境变量问题

由于 macOS LaunchAgent 不会自动继承 shell 环境变量，需要创建：

~/openclaw/.env

内容：

```
GEMINI_API_KEY=YOUR_KEY
https_proxy=http://127.0.0.1:7897
http_proxy=http://127.0.0.1:7897
all_proxy=http://127.0.0.1:7897
NO_PROXY=127.0.0.1,localhost,::1
```

然后重启 Gateway：

```
openclaw gateway stop
```

```
launchctl bootstrap gui/$UID ~/Library/LaunchAgents/ai.openclaw.gateway.plist
```

8 测试 OpenClaw

运行：

```
openclaw agent --agent main --message "hello"
```

成功输出示例：

```
Hello again, Asta!
```

9 使用 OpenClaw TUI

启动 AI 终端：

```
openclaw tui
```

即可进入交互式 AI 终端。

10 安装 Star Office UI

下载项目：

cd ~

git clone <https://github.com/ringhyacinth/Star-Office-UI.git>

进入目录：

cd Star-Office-UI

11 创建 Python 虚拟环境

python3.11 -m venv .venv

激活：

source .venv/bin/activate

升级 pip：

python -m pip install --upgrade pip

12 安装依赖

python -m pip install -r backend/requirements.txt

13 初始化状态文件

cp state.sample.json state.json

14 启动 UI 后端

cd backend

python app.py

成功输出：

Running on <http://127.0.0.1:19000>

15 打开像素 UI

浏览器访问：

<http://127.0.0.1:19000>

16 日常使用流程

Step1

确保 VPN / Clash 已启动

Step2

打开 AI

openclaw tui

Step3

启动 UI

cd ~/Star-Office-UI

source .venv/bin/activate

cd backend

python app.py

Step4

浏览器打开

<http://127.0.0.1:19000>

17 常见问题

17.1 fetch failed sending request

原因：

Gemini API 无法访问

解决：

确认代理

curl <https://generativelanguage.googleapis.com>

17.2 API_KEY_INVALID

原因：

API key 错误或无权限

解决：

重新生成 Gemini API key

17.3 command not found compdef

原因：

zsh 补全脚本加载顺序错误

解决：

注释：

source <(openclaw completion --shell zsh)

17.4 Python 语法错误 | None

原因：

Python 版本 < 3.10

解决：

安装 Python 3.11

17.5 Flask module not found

原因：

依赖未安装

解决：

pip install -r backend/requirements.txt

18 系统维护

停止 Gateway：

openclaw gateway stop

重新启动：

openclaw gateway start

19 项目未来扩展

可实现：

- Pixel AI 办公室
 - 实时 Agent 状态
 - Token 使用监控
 - 研究任务可视化
 - 自动论文分析
-

20 总结

完成后系统将成为一个本地 AI 工作台：

OpenClaw + Gemini + Pixel UI

具备：

- AI Agent
- 后台服务
- 可视化控制台

适用于：

- 研究 workflow
- 自动化任务
- AI 助手系统